# Ejecución condicional en Python

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

**Tabla de contenidos**

**Introduction** **3**

**1 Expresiones booleanas** **3**
1.1 Valores booleanos . . . 3
1.2 Función bool() . . . 3
1.3 Aplicación económica . . . 4

**2 Operadores de comparación** **4**
2.1 Operadores básicos . . . 4
2.2 Comparación de strings . . . 5
2.3 Comparación de listas . . . 6
2.4 Ejemplo aplicado: Clasificación de economías . . . 6

**3 Operadores lógicos** **7**
3.1 Operador AND . . . 7
3.2 Operador OR . . . 8
3.3 Operador NOT . . . 9
3.4 Combinación de operadores . . . 9

**4 Ejecución condicional simple** **10**
4.1 Sintaxis básica . . . 10
4.2 Ejemplos económicos . . . 10
4.3 Importancia de la indentación . . . 11

---

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

**5 Ejecución alternativa** **12**
5.1 Sintaxis if-else . . . 12
5.2 Ejemplos económicos . . . 12
5.3 Operador ternario . . . 13

**6 Condicionales encadenados** **14**
6.1 Sintaxis if-elif-else . . . 14
6.2 Ejemplos económicos . . . 14
6.3 Orden de evaluación . . . 16

**7 Condicionales anidados** **17**
7.1 Estructura básica . . . 17
7.2 Ejemplos económicos . . . 17
7.3 Alternativa a anidamiento excesivo . . . 20

**8 Evaluación de pertenencia e identidad** **20**
8.1 Operador in . . . 20
8.2 Operador is . . . 21
8.3 Función isinstance() . . . 22

**9 Guardianes: try y except** **23**
9.1 Estructura básica . . . 23
9.2 Capturar excepciones específicas . . . 24
9.3 Múltiples except . . . 24
9.4 Cláusula else y finally . . . 25
9.5 Aplicaciones económicas . . . 25
9.6 Obtener información del error . . . 27

**10 Evaluación de cortocircuito** **28**
10.1 Cortocircuito con AND . . . 28
10.2 Cortocircuito con OR . . . 29
10.3 Uso práctico en economía . . . 29

**11 Aplicaciones económicas** **31**
11.1 Aplicación 1: Sistema de clasificación de riesgo país . . . 31
11.2 Aplicación 2: Calculadora de impuestos progresivos . . . 35

**12 Ejercicios prácticos** **40**
12.1 Ejercicio 1: Sistema de evaluación de proyectos de inversión . . . 40
12.2 Ejercicio 2: Simulador de política monetaria . . . 43

**13 Conclusión** **48**

**14 Próximos pasos** **48**

**15 Recursos adicionales** **49**

**16 Publicaciones Similares** **49**

**Ejecución condicional en Python**

En esta cuarta guía exploraremos cómo los programas pueden tomar decisiones basadas en condiciones. La ejecución condicional es fundamental para crear análisis económicos automatizados que respondan a diferentes escenarios y situaciones.

## 1 Expresiones booleanas

Las expresiones booleanas son expresiones que evalúan a uno de dos valores posibles: True (verdadero) o False (falso). Son la base de la toma de decisiones en programación.

### 1.1 Valores booleanos

```
# Valores booleanos básicos
valor_verdadero = True
valor_falso = False

print(valor_verdadero)  # True
print(valor_falso)      # False
print(type(valor_verdadero))  # <class 'bool'>

# En economía
economia_en_crecimiento = True
hay_recesion = False
meta_inflacion_cumplida = True
```

```
True
False
<class 'bool'>
```

### 1.2 Función bool()

La función `bool()` convierte valores a booleanos:

```
# Valores que evalúan a False
print(bool(0))        # False
print(bool(0.0))      # False
print(bool(""))       # False (string vacío)
print(bool([]))       # False (lista vacía)
print(bool({}))       # False (diccionario vacío)
print(bool(None))     # False
print(bool(False))    # False

# Valores que evalúan a True
print(bool(1))        # True
print(bool(-1))       # True
print(bool(0.1))      # True
print(bool("texto"))  # True
print(bool([1, 2]))   # True
print(bool({'a': 1})) # True
```

```
False
False
False
False
False
False
False
True
True
True
True
True
True
```

### 1.3 Aplicación económica

```python
# Evaluar datos económicos
pib = 242632
inflacion = 3.5
desempleo = 7.2

# Convertir a booleanos
hay_datos_pib = bool(pib)
hay_datos_inflacion = bool(inflacion)

print(f"¿Hay datos de PIB? {hay_datos_pib}")
print(f"¿Hay datos de inflación? {hay_datos_inflacion}")

# Listas vacías como indicador
sectores_analizados = []
regiones_visitadas = ['Lima', 'Arequipa']

print(f"¿Se analizaron sectores? {bool(sectores_analizados)}")  # False
print(f"¿Se visitaron regiones? {bool(regiones_visitadas)}")    # True
```

```
¿Hay datos de PIB? True
¿Hay datos de inflación? True
¿Se analizaron sectores? False
¿Se visitaron regiones? True
```

## 2 Operadores de comparación

Los operadores de comparación comparan dos valores y retornan un booleano:

### 2.1 Operadores básicos

```python
# Datos económicos para comparar
pib_2022 = 240000
pib_2023 = 242632
inflacion_actual = 3.5
meta_inflacion = 3.0

# Igual a (==)
print(pib_2022 == pib_2023)  # False
print(5 == 5.0)              # True (compara valor, no tipo)

# Diferente de (!=)
print(pib_2022 != pib_2023)  # True
print(inflacion_actual != meta_inflacion)  # True

# Mayor que (>)
print(pib_2023 > pib_2022)   # True
print(inflacion_actual > meta_inflacion)  # True

# Menor que (<)
print(pib_2022 < pib_2023)   # True
print(meta_inflacion < inflacion_actual)  # True

# Mayor o igual que (>=)
print(pib_2023 >= pib_2022)  # True
print(pib_2023 >= 242632)    # True

# Menor o igual que (<=)
print(meta_inflacion <= inflacion_actual)  # True
print(meta_inflacion <= 3.0)  # True
```

```
False
True
True
True
True
True
True
True
True
True
True
True
True
```

### 2.2 Comparación de strings

```python
# Los strings se comparan lexicográficamente
pais_1 = "Argentina"
```

```
pais_2 = "Perú"
pais_3 = "Perú"

print(pais_1 == pais_2)  # False
print(pais_2 == pais_3)  # True
print(pais_1 < pais_2)   # True (orden alfabético)

# Sensibilidad a mayúsculas
sector_1 = "Minería"
sector_2 = "minería"

print(sector_1 == sector_2)  # False
print(sector_1.lower() == sector_2.lower())  # True
```

```
False
True
True
False
True
```

### 2.3 Comparación de listas

```
# Listas se comparan elemento por elemento
precios_2022 = [100, 105, 110]
precios_2023 = [100, 105, 110]
precios_2024 = [100, 105, 115]

print(precios_2022 == precios_2023)  # True
print(precios_2022 == precios_2024)  # False
print(precios_2022 < precios_2024)   # True (compara primer elemento diferente)

# Orden importa
lista_a = [1, 2, 3]
lista_b = [3, 2, 1]
print(lista_a == lista_b)  # False
```

```
True
False
True
False
```

### 2.4 Ejemplo aplicado: Clasificación de economías

```
# Clasificar países por PIB per cápita
pib_pc_peru = 7196
pib_pc_chile = 15400
pib_pc_argentina = 10600
```

```
umbral_pais_desarrollado = 12000

print(f"¿Perú es país desarrollado? {pib_pc_peru > umbral_pais_desarrollado}")
print(f"¿Chile es país desarrollado? {pib_pc_chile > umbral_pais_desarrollado}")
print(f"¿Argentina es país desarrollado? {pib_pc_argentina > umbral_pais_desarrollado

# Comparar entre países
print(f"\n¿Chile tiene mayor PIB per cápita que Perú? {pib_pc_chile > pib_pc_peru}")
print(f"¿Perú y Argentina tienen el mismo PIB per cápita? {pib_pc_peru == pib_pc_arge
```

```
¿Perú es país desarrollado? False
¿Chile es país desarrollado? True
¿Argentina es país desarrollado? False

¿Chile tiene mayor PIB per cápita que Perú? True
¿Perú y Argentina tienen el mismo PIB per cápita? False
```

## 3 Operadores lógicos

Los operadores lógicos combinan múltiples expresiones booleanas:

### 3.1 Operador AND

El operador and retorna True solo si ambas condiciones son verdaderas:

```
# Tabla de verdad de AND
print(True and True)    # True
print(True and False)   # False
print(False and True)   # False
print(False and False)  # False

# Aplicación económica
inflacion = 3.2
desempleo = 6.8
crecimiento = 2.5

# Economía estable: inflación baja Y desempleo bajo Y crecimiento positivo
inflacion_controlada = inflacion < 4.0
desempleo_bajo = desempleo < 8.0
crecimiento_positivo = crecimiento > 0

economia_estable = inflacion_controlada and desempleo_bajo and crecimiento_positivo
print(f"¿Economía estable? {economia_estable}")  # True

# Ejemplo: Aprobar un crédito
ingreso_mensual = 5000
antiguedad_laboral = 3  # años
historial_crediticio = "bueno"
```

```
ingreso_suficiente = ingreso_mensual >= 3000
estabilidad_laboral = antiguedad_laboral >= 2
buen_historial = historial_crediticio == "bueno"

aprobar_credito = ingreso_suficiente and estabilidad_laboral and buen_historial
print(f"¿Aprobar crédito? {aprobar_credito}")  # True
```

```
True
False
False
False
¿Economía estable? True
¿Aprobar crédito? True
```

### 3.2 Operador OR

El operador or retorna True si al menos una condición es verdadera:

```
# Tabla de verdad de OR
print(True or True)     # True
print(True or False)    # True
print(False or True)    # True
print(False or False)   # False

# Aplicación económica: Señales de alerta
inflacion = 5.5
desempleo = 9.2
deficit_fiscal = 4.8

inflacion_alta = inflacion > 5.0
desempleo_alto = desempleo > 8.0
deficit_alto = deficit_fiscal > 4.0

alerta_economica = inflacion_alta or desempleo_alto or deficit_alto
print(f"¿Hay alerta económica? {alerta_economica}")  # True

# Ejemplo: Elegibilidad para programa social
ingreso_familiar = 1200
numero_dependientes = 3
zona_rural = True

ingreso_bajo = ingreso_familiar < 1500
familia_numerosa = numero_dependientes >= 3

elegible = ingreso_bajo or familia_numerosa or zona_rural
print(f"¿Elegible para programa? {elegible}")  # True
```

```
True
```

```
True
True
False
¿Hay alerta económica? True
¿Elegible para programa? True
```

### 3.3 Operador NOT

El operador `not` invierte el valor booleano:

```python
# Tabla de verdad de NOT
print(not True)   # False
print(not False)  # True

# Aplicación económica
en_recesion = False
tiene_empleo = True
deuda_impagable = False

print(f"¿No está en recesión? {not en_recesion}")  # True
print(f"¿Está desempleado? {not tiene_empleo}")    # False
print(f"¿Deuda manejable? {not deuda_impagable}")  # True

# Ejemplo: Verificar condiciones negativas
cumple_meta_inflacion = False
supera_crecimiento_esperado = True

no_cumple_meta = not cumple_meta_inflacion
print(f"¿Necesita ajuste de política? {no_cumple_meta}")  # True
```

```
False
True
¿No está en recesión? True
¿Está desempleado? False
¿Deuda manejable? True
¿Necesita ajuste de política? True
```

### 3.4 Combinación de operadores

```python
# Los operadores se pueden combinar
# Precedencia: not > and > or

# Análisis de inversión
rentabilidad = 8.5
riesgo = "medio"
liquidez = "alta"
plazo_anos = 3
```

```
# Condiciones
rentabilidad_aceptable = rentabilidad > 6.0
riesgo_aceptable = riesgo in ["bajo", "medio"]
liquidez_aceptable = liquidez in ["media", "alta"]
plazo_adecuado = plazo_anos <= 5

# Decisión de inversión
invertir = (rentabilidad_aceptable and riesgo_aceptable) and \
           (liquidez_aceptable or plazo_adecuado)

print(f"¿Realizar inversión? {invertir}")

# Uso de paréntesis para claridad
inflacion = 4.2
desempleo = 7.5
reservas = 75000

situacion_critica = (inflacion > 5.0 and desempleo > 10.0) or \
                    (reservas < 50000)

print(f"¿Situación crítica? {situacion_critica}")  # False
```

```
¿Realizar inversión? True
¿Situación crítica? False
```

## 4 Ejecución condicional simple

La estructura `if` ejecuta código solo si una condición es verdadera:

### 4.1 Sintaxis básica

```
# Estructura básica
condicion = True

if condicion:
    print("La condición es verdadera")
    print("Este código se ejecuta")

# Sin indentación, no es parte del if
print("Este código siempre se ejecuta")
```

```
La condición es verdadera
Este código se ejecuta
Este código siempre se ejecuta
```

### 4.2 Ejemplos económicos

```python
# Ejemplo 1: Alerta de inflación
inflacion_mensual = 0.6

if inflacion_mensual > 0.5:
    print("ALERTA: Inflación mensual supera el 0.5%")
    print(f"Inflación registrada: {inflacion_mensual}%")
    print("Se recomienda revisar política monetaria")

# Ejemplo 2: Verificar meta de crecimiento
crecimiento_pib = 3.2
meta_crecimiento = 3.0

if crecimiento_pib >= meta_crecimiento:
    print("Meta de crecimiento alcanzada")
    print(f"Crecimiento: {crecimiento_pib}%")
    print(f"Meta: {meta_crecimiento}%")

# Ejemplo 3: Control de inventario
stock_actual = 8
stock_minimo = 10

if stock_actual < stock_minimo:
    print("ALERTA: Stock bajo el mínimo")
    faltante = stock_minimo - stock_actual
    print(f"Se requiere reponer {faltante} unidades")
```

```
ALERTA: Inflación mensual supera el 0.5%
Inflación registrada: 0.6%
Se recomienda revisar política monetaria
Meta de crecimiento alcanzada
Crecimiento: 3.2%
Meta: 3.0%
ALERTA: Stock bajo el mínimo
Se requiere reponer 2 unidades
```

### 4.3 Importancia de la indentación

```python
# Python usa indentación para delimitar bloques
x = 10

if x > 5:
    print("x es mayor que 5")
    print("Este también es parte del if")
print("Este no es parte del if")

# Error de indentación (comentado para evitar error)
# if x > 5:
```

```
# print("Error: no hay indentación")

# Indentación inconsistente (comentado)
# if x > 5:
#     print("4 espacios")
#       print("6 espacios - Error!")
```

```
x es mayor que 5
Este también es parte del if
Este no es parte del if
```

## 5 Ejecución alternativa

La estructura `if-else` ejecuta un bloque si la condición es verdadera, y otro si es falsa:

### 5.1 Sintaxis if-else

```
# Estructura básica
condicion = True

if condicion:
    print("Condición verdadera")
else:
    print("Condición falsa")
```

```
Condición verdadera
```

### 5.2 Ejemplos económicos

```
# Ejemplo 1: Clasificar inflación
inflacion = 3.5
meta = 3.0

if inflacion <= meta:
    print("Inflación bajo control")
    print(f"Inflación: {inflacion}%")
    print(f"Meta: {meta}%")
else:
    print("Inflación por encima de la meta")
    exceso = inflacion - meta
    print(f"Exceso: {exceso}%")
    print("Se requiere ajuste en política monetaria")

# Ejemplo 2: Determinar superávit o déficit
ingresos = 1500000
gastos = 1450000
```

```
if ingresos >= gastos:
    superavit = ingresos - gastos
    print(f"Superávit fiscal: S/ {superavit:,}")
    print("Situación fiscal favorable")
else:
    deficit = gastos - ingresos
    print(f"Déficit fiscal: S/ {deficit:,}")
    print("Se requiere ajuste fiscal")

# Ejemplo 3: Calificación de riesgo crediticio
puntaje_crediticio = 720
umbral = 700

if puntaje_crediticio >= umbral:
    tasa_interes = 8.5
    print("Crédito aprobado")
    print(f"Tasa de interés: {tasa_interes}%")
else:
    print("Crédito denegado")
    print("Puntaje insuficiente")
    print(f"Puntaje actual: {puntaje_crediticio}")
    print(f"Puntaje requerido: {umbral}")
```

```
Inflación por encima de la meta
Exceso: 0.5%
Se requiere ajuste en política monetaria
Superávit fiscal: S/ 50,000
Situación fiscal favorable
Crédito aprobado
Tasa de interés: 8.5%
```

### 5.3 Operador ternario

Python permite escribir if-else en una línea:

```
# Sintaxis: valor_si_true if condicion else valor_si_false

# Ejemplo 1: Clasificar país
pib_per_capita = 15000
clasificacion = "Desarrollado" if pib_per_capita > 12000 else "En desarrollo"
print(f"Clasificación: {clasificacion}")

# Ejemplo 2: Determinar signo
saldo = 5000
signo = "+" if saldo >= 0 else "-"
print(f"Saldo: {signo}S/ {abs(saldo):,}")

# Ejemplo 3: Calcular descuento
```

```python
precio = 1000
cantidad = 15
descuento = 0.10 if cantidad >= 10 else 0
precio_final = precio * (1 - descuento)
print(f"Precio final: S/ {precio_final:.2f}")
```

```
Clasificación: Desarrollado
Saldo: +S/ 5,000
Precio final: S/ 900.00
```

## 6 Condicionales encadenados

La estructura `if-elif-else` permite evaluar múltiples condiciones en secuencia:

### 6.1 Sintaxis if-elif-else

```python
# Estructura básica
x = 10

if x < 0:
    print("Negativo")
elif x == 0:
    print("Cero")
elif x < 10:
    print("Positivo menor que 10")
else:
    print("10 o mayor")
```

```
10 o mayor
```

### 6.2 Ejemplos económicos

```python
# Ejemplo 1: Clasificación de inflación
inflacion = 4.5

if inflacion < 2.0:
    clasificacion = "Muy baja"
    comentario = "Riesgo de deflación"
elif inflacion < 3.0:
    clasificacion = "Baja"
    comentario = "Dentro del objetivo"
elif inflacion < 4.0:
    clasificacion = "Moderada"
    comentario = "Ligeramente por encima del objetivo"
elif inflacion < 6.0:
    clasificacion = "Alta"
    comentario = "Requiere medidas correctivas"
```

```
else:
    clasificacion = "Muy alta"
    comentario = "Situación crítica"

print(f"Inflación: {inflacion}%")
print(f"Clasificación: {clasificacion}")
print(f"Comentario: {comentario}")

# Ejemplo 2: Escala de riesgo crediticio
puntaje = 680

if puntaje >= 800:
    riesgo = "Muy bajo"
    tasa = 7.5
elif puntaje >= 740:
    riesgo = "Bajo"
    tasa = 9.0
elif puntaje >= 670:
    riesgo = "Medio"
    tasa = 12.0
elif puntaje >= 580:
    riesgo = "Alto"
    tasa = 16.0
else:
    riesgo = "Muy alto"
    tasa = None

print(f"Puntaje crediticio: {puntaje}")
print(f"Nivel de riesgo: {riesgo}")
if tasa:
    print(f"Tasa de interés: {tasa}%")
else:
    print("Crédito no disponible")

# Ejemplo 3: Clasificación de países por ingreso
ingreso_per_capita = 4500

if ingreso_per_capita < 1045:
    categoria = "Ingreso bajo"
    grupo = "Países menos desarrollados"
elif ingreso_per_capita < 4095:
    categoria = "Ingreso medio-bajo"
    grupo = "Economías emergentes"
elif ingreso_per_capita < 12695:
    categoria = "Ingreso medio-alto"
    grupo = "Economías en transición"
else:
```

```
    categoria = "Ingreso alto"
    grupo = "Países desarrollados"

print(f"Ingreso per cápita: USD {ingreso_per_capita:,}")
print(f"Categoría: {categoria}")
print(f"Grupo: {grupo}")
```

```
Inflación: 4.5%
Clasificación: Alta
Comentario: Requiere medidas correctivas
Puntaje crediticio: 680
Nivel de riesgo: Medio
Tasa de interés: 12.0%
Ingreso per cápita: USD 4,500
Categoría: Ingreso medio-alto
Grupo: Economías en transición
```

### 6.3 Orden de evaluación

```
# El orden importa: se evalúa de arriba hacia abajo
# Solo se ejecuta el primer bloque que cumple la condición

nota = 85

if nota >= 90:
    calificacion = "Excelente"
elif nota >= 80:
    calificacion = "Muy bueno"
elif nota >= 70:
    calificacion = "Bueno"
elif nota >= 60:
    calificacion = "Regular"
else:
    calificacion = "Insuficiente"

print(f"Nota: {nota}")
print(f"Calificación: {calificacion}")

# Si cambiamos el orden, el resultado puede ser diferente
# (Ejemplo de lo que NO hacer)
if nota >= 60:  # Este se evalúa primero
    calificacion_incorrecta = "Regular"
elif nota >= 90:  # Nunca se alcanza
    calificacion_incorrecta = "Excelente"

print(f"Calificación incorrecta: {calificacion_incorrecta}")  # Regular
```

```
Nota: 85
Calificación: Muy bueno
Calificación incorrecta: Regular
```

## 7 Condicionales anidados

Los condicionales pueden contener otros condicionales dentro:

### 7.1 Estructura básica

```python
# Condicional anidado simple
x = 10

if x > 0:
    print("x es positivo")
    if x > 5:
        print("x es mayor que 5")
    else:
        print("x es menor o igual a 5")
else:
    print("x es cero o negativo")
```

```
x es positivo
x es mayor que 5
```

### 7.2 Ejemplos económicos

```python
# Ejemplo 1: Análisis de elegibilidad para crédito hipotecario
ingreso_mensual = 8000
ahorro_inicial = 150000
historial_crediticio = "bueno"
precio_vivienda = 500000

print("ANÁLISIS DE ELEGIBILIDAD - CRÉDITO HIPOTECARIO")
print("=" * 60)

# Primera condición: ingreso mínimo
if ingreso_mensual >= 5000:
    print(f"  Ingreso mensual adecuado: S/ {ingreso_mensual:,}")

    # Segunda condición: ahorro inicial
    ahorro_minimo = precio_vivienda * 0.20

    if ahorro_inicial >= ahorro_minimo:
        print(f"  Ahorro inicial suficiente: S/ {ahorro_inicial:,}")
        print(f"   (Requerido: S/ {ahorro_minimo:,})")

        # Tercera condición: historial crediticio
```

```
        if historial_crediticio == "excelente":
            tasa_interes = 7.5
            print(f"  Historial crediticio excelente")
            print(f"  Tasa de interés: {tasa_interes}%")
        elif historial_crediticio == "bueno":
            tasa_interes = 8.5
            print(f"  Historial crediticio bueno")
            print(f"  Tasa de interés: {tasa_interes}%")
        else:
            print("  Historial crediticio insuficiente")
            tasa_interes = None

        # Decisión final
        if tasa_interes:
            monto_credito = precio_vivienda - ahorro_inicial
            print(f"\n{'='*60}")
            print(f"CRÉDITO APROBADO")
            print(f"Monto del crédito: S/ {monto_credito:,}")
            print(f"Tasa de interés: {tasa_interes}%")
        else:
            print(f"\nCrédito denegado por historial crediticio")
    else:
        faltante = ahorro_minimo - ahorro_inicial
        print(f"  Ahorro inicial insuficiente")
        print(f"  Ahorro actual: S/ {ahorro_inicial:,}")
        print(f"  Ahorro requerido: S/ {ahorro_minimo:,}")
        print(f"  Faltante: S/ {faltante:,}")
else:
    print(f"  Ingreso mensual insuficiente: S/ {ingreso_mensual:,}")
    print(f"  Ingreso mínimo requerido: S/ 5,000")

# Ejemplo 2: Clasificación de empresa por tamaño y sector
ventas_anuales = 8500000  # en soles
num_trabajadores = 85
sector = "manufactura"

print("\n" + "="*60)
print("CLASIFICACIÓN DE EMPRESA")
print("="*60)

if sector == "manufactura":
    print(f"Sector: Manufactura")

    if ventas_anuales <= 1700000:
        if num_trabajadores <= 10:
            clasificacion = "Microempresa"
        else:
```

```
            clasificacion = "Pequeña empresa"
    elif ventas_anuales <= 17000000:
        if num_trabajadores <= 100:
            clasificacion = "Pequeña empresa"
        else:
            clasificacion = "Mediana empresa"
    else:
        clasificacion = "Gran empresa"

elif sector == "servicios":
    print(f"Sector: Servicios")

    if ventas_anuales <= 1100000:
        clasificacion = "Microempresa"
    elif ventas_anuales <= 11000000:
        clasificacion = "Pequeña empresa"
    elif ventas_anuales <= 110000000:
        clasificacion = "Mediana empresa"
    else:
        clasificacion = "Gran empresa"
else:
    clasificacion = "Sector no clasificado"

print(f"Ventas anuales: S/ {ventas_anuales:,}")
print(f"Número de trabajadores: {num_trabajadores}")
print(f"Clasificación: {clasificacion}")
```

```
ANÁLISIS DE ELEGIBILIDAD - CRÉDITO HIPOTECARIO
============================================================
  Ingreso mensual adecuado: S/ 8,000
  Ahorro inicial suficiente: S/ 150,000
   (Requerido: S/ 100,000.0)
  Historial crediticio bueno
   Tasa de interés: 8.5%

============================================================
CRÉDITO APROBADO
Monto del crédito: S/ 350,000
Tasa de interés: 8.5%

============================================================
CLASIFICACIÓN DE EMPRESA
============================================================
Sector: Manufactura
Ventas anuales: S/ 8,500,000
Número de trabajadores: 85
Clasificación: Pequeña empresa
```

### 7.3 Alternativa a anidamiento excesivo

Los condicionales muy anidados pueden dificultar la lectura. Es mejor usar `elif`:

```
# Evitar anidamiento excesivo
ingreso = 5000
credito = "bueno"
empleo = "estable"

# Mal: muy anidado
if ingreso > 3000:
    if credito == "bueno":
        if empleo == "estable":
            print("Crédito aprobado")
        else:
            print("Crédito denegado: empleo inestable")
    else:
        print("Crédito denegado: mal historial")
else:
    print("Crédito denegado: ingreso bajo")

# Mejor: condiciones combinadas
if ingreso <= 3000:
    print("Crédito denegado: ingreso bajo")
elif credito != "bueno":
    print("Crédito denegado: mal historial")
elif empleo != "estable":
    print("Crédito denegado: empleo inestable")
else:
    print("Crédito aprobado")

# Aún mejor: usar operadores lógicos
if ingreso > 3000 and credito == "bueno" and empleo == "estable":
    print("Crédito aprobado")
else:
    print("Crédito denegado")
```

```
Crédito aprobado
Crédito aprobado
Crédito aprobado
```

## 8 Evaluación de pertenencia e identidad

### 8.1 Operador in

Verifica si un elemento está en una secuencia:

```
# Verificar en listas
sectores_primarios = ['Agricultura', 'Pesca', 'Minería', 'Petróleo']
```

```
sector_analizar = 'Minería'

if sector_analizar in sectores_primarios:
    print(f"{sector_analizar} es un sector primario")
else:
    print(f"{sector_analizar} no es un sector primario")

# Verificar en strings
descripcion = "Crecimiento del Producto Bruto Interno"
if "PIB" in descripcion or "Producto Bruto Interno" in descripcion:
    print("El texto menciona el PIB")

# Verificar en diccionarios (verifica claves)
indicadores = {
    'pib': 242632,
    'inflacion': 3.5,
    'desempleo': 7.2
}

if 'pib' in indicadores:
    print(f"PIB disponible: {indicadores['pib']}")

if 'reservas' not in indicadores:
    print("No hay datos de reservas internacionales")

# Verificar en rangos
edad = 25
if edad in range(18, 65):
    print("En edad de trabajar")
```

```
Minería es un sector primario
El texto menciona el PIB
PIB disponible: 242632
No hay datos de reservas internacionales
En edad de trabajar
```

### 8.2 Operador is

Verifica identidad de objetos (mismo objeto en memoria):

```
# Comparar identidad vs igualdad
a = [1, 2, 3]
b = [1, 2, 3]
c = a

# Igualdad de valores
print(a == b)  # True (mismo contenido)
print(a == c)  # True (mismo contenido)
```

```python
# Identidad de objetos
print(a is b)  # False (objetos diferentes en memoria)
print(a is c)  # True (mismo objeto)

# Uso con None
valor = None
if valor is None:
    print("Valor no definido")

# Con booleanos
activo = True
if activo is True:
    print("Estado activo")

# Mejor práctica: comparar con None usando 'is'
datos = None
if datos is None:
    print("No hay datos disponibles")
else:
    print(f"Datos: {datos}")
```

```
True
True
False
True
Valor no definido
Estado activo
No hay datos disponibles
```

### 8.3 Función isinstance()

Verifica el tipo de un objeto:

```python
# Verificar tipos
pib = 242632
inflacion = 3.5
pais = "Perú"
sectores = ['Minería', 'Agricultura']
datos = {'pib': 242632, 'inflacion': 3.5}

print(isinstance(pib, int))           # True
print(isinstance(inflacion, float))   # True
print(isinstance(pais, str))          # True
print(isinstance(sectores, list))     # True
print(isinstance(datos, dict))        # True

# Verificar múltiples tipos
valor = 42
```

```python
if isinstance(valor, (int, float)):
    print("Es un número")

# Aplicación: validar entrada
def calcular_interes(capital, tasa, tiempo):
    """Calcula interés simple"""

    # Validar tipos
    if not isinstance(capital, (int, float)):
        return "Error: capital debe ser numérico"

    if not isinstance(tasa, (int, float)):
        return "Error: tasa debe ser numérica"

    if not isinstance(tiempo, (int, float)):
        return "Error: tiempo debe ser numérico"

    # Calcular
    interes = capital * tasa * tiempo
    return interes

# Probar función
resultado1 = calcular_interes(10000, 0.08, 3)
print(f"Interés: S/ {resultado1:,.2f}")

resultado2 = calcular_interes("10000", 0.08, 3)
print(resultado2)  # Error: capital debe ser numérico
```

```
True
True
True
True
True
Es un número
Interés: S/ 2,400.00
Error: capital debe ser numérico
```

## 9 Guardianes: try y except

El manejo de excepciones permite capturar y manejar errores sin que el programa se detenga:

### 9.1 Estructura básica

```python
# Sin manejo de errores (genera error)
# numero = int("abc")  # ValueError

# Con manejo de errores
```

```
try:
    numero = int("abc")
    print(numero)
except:
    print("Error: no se pudo convertir a número")

print("El programa continúa")
```

```
Error: no se pudo convertir a número
El programa continúa
```

### 9.2 Capturar excepciones específicas

```
# Capturar tipos específicos de errores
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("Error: división por cero")

try:
    lista = [1, 2, 3]
    print(lista[5])
except IndexError:
    print("Error: índice fuera de rango")

try:
    diccionario = {'a': 1, 'b': 2}
    print(diccionario['c'])
except KeyError:
    print("Error: clave no existe")

try:
    numero = int("texto")
except ValueError:
    print("Error: valor no numérico")
```

```
Error: división por cero
Error: índice fuera de rango
Error: clave no existe
Error: valor no numérico
```

### 9.3 Múltiples except

```
# Manejar diferentes errores
def dividir(a, b):
    try:
        resultado = a / b
```

```
        return resultado
    except ZeroDivisionError:
        return "Error: no se puede dividir por cero"
    except TypeError:
        return "Error: los valores deben ser numéricos"

print(dividir(10, 2))      # 5.0
print(dividir(10, 0))      # Error: no se puede dividir por cero
print(dividir(10, "2"))    # Error: los valores deben ser numéricos
```

```
5.0
Error: no se puede dividir por cero
Error: los valores deben ser numéricos
```

### 9.4 Cláusula else y finally

```
# else: se ejecuta si no hay error
# finally: siempre se ejecuta

try:
    numero = int(input("Ingrese un número: "))
except ValueError:
    print("Error: debe ingresar un número")
else:
    print(f"Número ingresado: {numero}")
    cuadrado = numero ** 2
    print(f"Cuadrado: {cuadrado}")
finally:
    print("Fin del proceso")
```

### 9.5 Aplicaciones económicas

```
# Ejemplo 1: Validar entrada de datos
def calcular_tasa_crecimiento(valor_inicial, valor_final):
    """Calcula la tasa de crecimiento"""

    try:
        # Intentar convertir a float
        v_inicial = float(valor_inicial)
        v_final = float(valor_final)

        # Validar valores positivos
        if v_inicial <= 0:
            return "Error: el valor inicial debe ser positivo"

        # Calcular tasa
```

```
        tasa = ((v_final - v_inicial) / v_inicial) * 100
        return tasa

    except ValueError:
        return "Error: los valores deben ser numéricos"
    except ZeroDivisionError:
        return "Error: el valor inicial no puede ser cero"

# Probar función
print(calcular_tasa_crecimiento(100, 110))      # 10.0
print(calcular_tasa_crecimiento("100", "110"))  # 10.0
print(calcular_tasa_crecimiento("abc", 110))    # Error
print(calcular_tasa_crecimiento(0, 110))        # Error

# Ejemplo 2: Leer datos de diccionario de forma segura
datos_pais = {
    'nombre': 'Perú',
    'pib': 242632,
    'poblacion': 33715471
}

def obtener_indicador(pais, indicador):
    """Obtiene un indicador del diccionario de forma segura"""

    try:
        valor = pais[indicador]
        return valor
    except KeyError:
        return f"Indicador '{indicador}' no disponible"
    except TypeError:
        return "Error: formato de datos incorrecto"

print(obtener_indicador(datos_pais, 'pib'))         # 242632
print(obtener_indicador(datos_pais, 'inflacion'))   # No disponible

# Ejemplo 3: Cálculo con validación
def calcular_pib_per_capita(pib, poblacion):
    """Calcula PIB per cápita con manejo de errores"""

    try:
        pib_pc = (pib / poblacion) * 1000000
        return pib_pc
    except ZeroDivisionError:
        return "Error: población no puede ser cero"
    except TypeError:
        return "Error: valores deben ser numéricos"
    except Exception as e:
```

```
        return f"Error inesperado: {e}"

print(calcular_pib_per_capita(242632, 33715471))    # 7196.21
print(calcular_pib_per_capita(242632, 0))           # Error: población...
print(calcular_pib_per_capita("242632", 33715471))  # Error: valores...
```

```
10.0
10.0
Error: los valores deben ser numéricos
Error: el valor inicial debe ser positivo
242632
Indicador 'inflacion' no disponible
7196.458860088296
Error: población no puede ser cero
Error: valores deben ser numéricos
```

### 9.6 Obtener información del error

```
# Capturar el mensaje de error
try:
    resultado = 10 / 0
except ZeroDivisionError as error:
    print(f"Error capturado: {error}")
    print(f"Tipo de error: {type(error).__name__}")

# Aplicación: log de errores
def procesar_datos(datos):
    errores = []

    for i, valor in enumerate(datos):
        try:
            resultado = 100 / valor
            print(f"Índice {i}: {resultado:.2f}")
        except Exception as e:
            mensaje_error = f"Índice {i}: {type(e).__name__} - {e}"
            errores.append(mensaje_error)

    if errores:
        print("\nErrores encontrados:")
        for error in errores:
            print(f"  - {error}")

datos_prueba = [10, 5, 0, 2, "a", 4]
procesar_datos(datos_prueba)
```

```
Error capturado: division by zero
Tipo de error: ZeroDivisionError
```

```
Índice 0: 10.00
Índice 1: 20.00
Índice 3: 50.00
Índice 5: 25.00

Errores encontrados:
  - Índice 2: ZeroDivisionError - division by zero
  - Índice 4: TypeError - unsupported operand type(s) for /: 'int' and 'str'
```

## 10 Evaluación de cortocircuito

Python evalúa expresiones booleanas de izquierda a derecha y se detiene cuando el resultado es determinado:

### 10.1 Cortocircuito con AND

```python
# and: se detiene en el primer False
def funcion_a():
    print("Ejecutando función A")
    return False

def funcion_b():
    print("Ejecutando función B")
    return True

# funcion_b nunca se ejecuta
resultado = funcion_a() and funcion_b()
print(f"Resultado: {resultado}")
# Salida:
# Ejecutando función A
# Resultado: False

# Aplicación: evitar errores
datos = []
# Sin cortocircuito, esto daría error:
# if datos[0] > 0:  # IndexError

# Con cortocircuito, es seguro:
if len(datos) > 0 and datos[0] > 0:
    print("Primer elemento es positivo")
else:
    print("Lista vacía o primer elemento no positivo")
```

```
Ejecutando función A
Resultado: False
Lista vacía o primer elemento no positivo
```

### 10.2 Cortocircuito con OR

```
# or: se detiene en el primer True
def funcion_c():
    print("Ejecutando función C")
    return True

def funcion_d():
    print("Ejecutando función D")
    return False

# funcion_d nunca se ejecuta
resultado = funcion_c() or funcion_d()
print(f"Resultado: {resultado}")
# Salida:
# Ejecutando función C
# Resultado: True

# Aplicación: valores por defecto
nombre = ""
nombre_mostrar = nombre or "Sin nombre"
print(nombre_mostrar)  # "Sin nombre"

pais = "Perú"
pais_mostrar = pais or "No especificado"
print(pais_mostrar)  # "Perú"
```

```
Ejecutando función C
Resultado: True
Sin nombre
Perú
```

### 10.3 Uso práctico en economía

```
# Ejemplo 1: Validación eficiente
def analizar_empresa(datos):
    """Analiza datos de empresa con validación eficiente"""

    # Verificaciones en orden de eficiencia
    # Se detiene en la primera que falla
    if not datos:
        return "Error: no hay datos"

    if 'ingresos' not in datos:
        return "Error: faltan ingresos"

    if 'gastos' not in datos:
```

```
        return "Error: faltan gastos"

    # Si llegamos aquí, todos los datos están
    utilidad = datos['ingresos'] - datos['gastos']
    margen = (utilidad / datos['ingresos']) * 100

    return f"Utilidad: S/ {utilidad:,}, Margen: {margen:.2f}%"

# Pruebas
print(analizar_empresa({}))
print(analizar_empresa({'ingresos': 100000}))
print(analizar_empresa({'ingresos': 100000, 'gastos': 80000}))

# Ejemplo 2: Acceso seguro a datos anidados
pais_data = {
    'nombre': 'Perú',
    'economia': {
        'pib': 242632,
        'sectores': {
            'mineria': 30000
        }
    }
}

# Acceso con cortocircuito
tiene_mineria = pais_data and \
                'economia' in pais_data and \
                'sectores' in pais_data['economia'] and \
                'mineria' in pais_data['economia']['sectores']

if tiene_mineria:
    valor_mineria = pais_data['economia']['sectores']['mineria']
    print(f"Minería: {valor_mineria}")
else:
    print("Datos de minería no disponibles")

# Ejemplo 3: Búsqueda eficiente
def buscar_indicador(indicadores, nombres_posibles):
    """Busca un indicador por varios nombres posibles"""

    # Se detiene en el primer nombre encontrado
    for nombre in nombres_posibles:
        if nombre in indicadores:
            return indicadores[nombre]

    return None
```

```
indicadores = {
    'producto_bruto_interno': 242632,
    'tasa_inflacion': 3.5,
    'tasa_desempleo': 7.2
}

# Buscar PIB con nombres alternativos
pib = buscar_indicador(indicadores, ['pib', 'producto_bruto_interno', 'gdp'])
print(f"PIB encontrado: {pib}")
```

```
Error: no hay datos
Error: faltan gastos
Utilidad: S/ 20,000, Margen: 20.00%
Minería: 30000
PIB encontrado: 242632
```

## 11 Aplicaciones económicas

### 11.1 Aplicación 1: Sistema de clasificación de riesgo país

```
def clasificar_riesgo_pais(datos):
    """
    Clasifica el riesgo país según múltiples indicadores
    """

    print("SISTEMA DE CLASIFICACIÓN DE RIESGO PAÍS")
    print("=" * 60)

    try:
        # Extraer indicadores
        pib_crecimiento = datos.get('pib_crecimiento', 0)
        inflacion = datos.get('inflacion', 0)
        deficit_fiscal = datos.get('deficit_fiscal', 0)
        deuda_pib = datos.get('deuda_pib', 0)
        reservas_meses_importacion = datos.get('reservas', 0)

        # Inicializar puntaje
        puntaje = 0
        detalles = []

        # Evaluar crecimiento económico
        if pib_crecimiento >= 4.0:
            puntaje += 3
            detalles.append("  Crecimiento sólido (3 puntos)")
        elif pib_crecimiento >= 2.0:
            puntaje += 2
            detalles.append("  Crecimiento moderado (2 puntos)")
        elif pib_crecimiento >= 0:
```

```
            puntaje += 1
            detalles.append("  Crecimiento débil (1 punto)")
        else:
            detalles.append("  Recesión (0 puntos)")

        # Evaluar inflación
        if inflacion <= 3.0:
            puntaje += 3
            detalles.append("  Inflación controlada (3 puntos)")
        elif inflacion <= 5.0:
            puntaje += 2
            detalles.append("  Inflación moderada (2 puntos)")
        elif inflacion <= 10.0:
            puntaje += 1
            detalles.append("  Inflación alta (1 punto)")
        else:
            detalles.append("  Inflación muy alta (0 puntos)")

        # Evaluar balance fiscal
        if deficit_fiscal <= 2.0:
            puntaje += 2
            detalles.append("  Balance fiscal saludable (2 puntos)")
        elif deficit_fiscal <= 4.0:
            puntaje += 1
            detalles.append("  Déficit moderado (1 punto)")
        else:
            detalles.append("  Déficit elevado (0 puntos)")

        # Evaluar deuda pública
        if deuda_pib <= 40.0:
            puntaje += 2
            detalles.append("  Deuda baja (2 puntos)")
        elif deuda_pib <= 60.0:
            puntaje += 1
            detalles.append("  Deuda moderada (1 punto)")
        else:
            detalles.append("  Deuda alta (0 puntos)")

        # Evaluar reservas internacionales
        if reservas_meses_importacion >= 6.0:
            puntaje += 2
            detalles.append("  Reservas sólidas (2 puntos)")
        elif reservas_meses_importacion >= 3.0:
            puntaje += 1
            detalles.append("  Reservas adecuadas (1 punto)")
        else:
            detalles.append("  Reservas bajas (0 puntos)")
```

```
        # Determinar clasificación
        if puntaje >= 11:
            clasificacion = "AAA"
            descripcion = "Riesgo muy bajo"
            spread = 100
        elif puntaje >= 9:
            clasificacion = "AA"
            descripcion = "Riesgo bajo"
            spread = 150
        elif puntaje >= 7:
            clasificacion = "A"
            descripcion = "Riesgo moderado"
            spread = 250
        elif puntaje >= 5:
            clasificacion = "BBB"
            descripcion = "Riesgo medio"
            spread = 400
        elif puntaje >= 3:
            clasificacion = "BB"
            descripcion = "Riesgo alto"
            spread = 600
        else:
            clasificacion = "B"
            descripcion = "Riesgo muy alto"
            spread = 1000

        # Mostrar resultados
        print(f"\nPaís: {datos.get('nombre', 'No especificado')}")
        print(f"\nIndicadores:")
        print(f"  Crecimiento PIB: {pib_crecimiento:+.1f}%")
        print(f"  Inflación: {inflacion:.1f}%")
        print(f"  Déficit fiscal: {deficit_fiscal:.1f}% del PIB")
        print(f"  Deuda pública: {deuda_pib:.1f}% del PIB")
        print(f"  Reservas: {reservas_meses_importacion:.1f} meses de importación")

        print(f"\nEvaluación:")
        for detalle in detalles:
            print(f"  {detalle}")

        print(f"\n{'='*60}")
        print(f"Puntaje total: {puntaje}/12")
        print(f"Clasificación: {clasificacion}")
        print(f"Descripción: {descripcion}")
        print(f"Spread estimado: {spread} puntos básicos")
        print(f"{'='*60}")

        return clasificacion
```

```
    except Exception as e:
        return f"Error en el análisis: {e}"

# Ejemplo de uso
peru_datos = {
    'nombre': 'Perú',
    'pib_crecimiento': 2.7,
    'inflacion': 3.5,
    'deficit_fiscal': 2.8,
    'deuda_pib': 35.0,
    'reservas': 8.5
}

clasificacion = clasificar_riesgo_pais(peru_datos)

# Comparar con otro país
venezuela_datos = {
    'nombre': 'Venezuela',
    'pib_crecimiento': -5.0,
    'inflacion': 1500.0,
    'deficit_fiscal': 15.0,
    'deuda_pib': 150.0,
    'reservas': 0.5
}

print("\n")
clasificacion_vzla = clasificar_riesgo_pais(venezuela_datos)
```

```
SISTEMA DE CLASIFICACIÓN DE RIESGO PAÍS
============================================================

País: Perú

Indicadores:
  Crecimiento PIB: +2.7%
  Inflación: 3.5%
  Déficit fiscal: 2.8% del PIB
  Deuda pública: 35.0% del PIB
  Reservas: 8.5 meses de importación

Evaluación:
    Crecimiento moderado (2 puntos)
    Inflación moderada (2 puntos)
    Déficit moderado (1 punto)
    Deuda baja (2 puntos)
    Reservas sólidas (2 puntos)
```

```
============================================================
Puntaje total: 9/12
Clasificación: AA
Descripción: Riesgo bajo
Spread estimado: 150 puntos básicos
============================================================


SISTEMA DE CLASIFICACIÓN DE RIESGO PAÍS
============================================================

País: Venezuela

Indicadores:
  Crecimiento PIB: -5.0%
  Inflación: 1500.0%
  Déficit fiscal: 15.0% del PIB
  Deuda pública: 150.0% del PIB
  Reservas: 0.5 meses de importación

Evaluación:
    Recesión (0 puntos)
    Inflación muy alta (0 puntos)
    Déficit elevado (0 puntos)
    Deuda alta (0 puntos)
    Reservas bajas (0 puntos)

============================================================
Puntaje total: 0/12
Clasificación: B
Descripción: Riesgo muy alto
Spread estimado: 1000 puntos básicos
============================================================
```

### 11.2 Aplicación 2: Calculadora de impuestos progresivos

```python
def calcular_impuesto_renta(ingreso_anual):
    """
    Calcula el impuesto a la renta con sistema progresivo
    (Basado en el sistema peruano simplificado)
    """

    print("CÁLCULO DE IMPUESTO A LA RENTA")
    print("=" * 60)

    try:
        # Validar entrada
        if not isinstance(ingreso_anual, (int, float)):
```

```
            return "Error: el ingreso debe ser numérico"

        if ingreso_anual < 0:
            return "Error: el ingreso no puede ser negativo"

        # Tramos de impuesto (valores en UIT)
        UIT = 5150  # UIT para 2026 (valor hipotético)

        # Definir tramos
        tramo1_limite = 5 * UIT      # 0-5 UIT: 8%
        tramo2_limite = 20 * UIT     # 5-20 UIT: 14%
        tramo3_limite = 35 * UIT     # 20-35 UIT: 17%
        tramo4_limite = 45 * UIT     # 35-45 UIT: 20%
        # Más de 45 UIT: 30%

        # Calcular impuesto
        impuesto = 0
        ingreso_restante = ingreso_anual
        desglose = []

        # Tramo 1: 0-5 UIT (8%)
        if ingreso_restante > 0:
            monto_tramo = min(ingreso_restante, tramo1_limite)
            impuesto_tramo = monto_tramo * 0.08
            impuesto += impuesto_tramo
            ingreso_restante -= monto_tramo

            if monto_tramo > 0:
                desglose.append({
                    'tramo': 'Tramo 1 (0-5 UIT)',
                    'base': monto_tramo,
                    'tasa': 0.08,
                    'impuesto': impuesto_tramo
                })

        # Tramo 2: 5-20 UIT (14%)
        if ingreso_restante > 0:
            monto_tramo = min(ingreso_restante, tramo2_limite - tramo1_limite)
            impuesto_tramo = monto_tramo * 0.14
            impuesto += impuesto_tramo
            ingreso_restante -= monto_tramo

            if monto_tramo > 0:
                desglose.append({
                    'tramo': 'Tramo 2 (5-20 UIT)',
                    'base': monto_tramo,
                    'tasa': 0.14,
```

```
                'impuesto': impuesto_tramo
            })

        # Tramo 3: 20-35 UIT (17%)
        if ingreso_restante > 0:
            monto_tramo = min(ingreso_restante, tramo3_limite - tramo2_limite)
            impuesto_tramo = monto_tramo * 0.17
            impuesto += impuesto_tramo
            ingreso_restante -= monto_tramo

            if monto_tramo > 0:
                desglose.append({
                    'tramo': 'Tramo 3 (20-35 UIT)',
                    'base': monto_tramo,
                    'tasa': 0.17,
                    'impuesto': impuesto_tramo
                })

        # Tramo 4: 35-45 UIT (20%)
        if ingreso_restante > 0:
            monto_tramo = min(ingreso_restante, tramo4_limite - tramo3_limite)
            impuesto_tramo = monto_tramo * 0.20
            impuesto += impuesto_tramo
            ingreso_restante -= monto_tramo

            if monto_tramo > 0:
                desglose.append({
                    'tramo': 'Tramo 4 (35-45 UIT)',
                    'base': monto_tramo,
                    'tasa': 0.20,
                    'impuesto': impuesto_tramo
                })

        # Tramo 5: Más de 45 UIT (30%)
        if ingreso_restante > 0:
            monto_tramo = ingreso_restante
            impuesto_tramo = monto_tramo * 0.30
            impuesto += impuesto_tramo

            desglose.append({
                'tramo': 'Tramo 5 (>45 UIT)',
                'base': monto_tramo,
                'tasa': 0.30,
                'impuesto': impuesto_tramo
            })

        # Calcular tasa efectiva
```

```
        tasa_efectiva = (impuesto / ingreso_anual) * 100 if ingreso_anual > 0 else 0
        ingreso_neto = ingreso_anual - impuesto

        # Mostrar resultados
        print(f"\nIngreso bruto anual: S/ {ingreso_anual:,.2f}")
        print(f"UIT vigente: S/ {UIT:,.2f}")
        print(f"\nDesglose por tramos:")
        print("-" * 60)

        for item in desglose:
            print(f"\n{item['tramo']}")
            print(f"  Base imponible: S/ {item['base']:,.2f}")
            print(f"  Tasa: {item['tasa']*100:.0f}%")
            print(f"  Impuesto: S/ {item['impuesto']:,.2f}")

        print(f"\n{'='*60}")
        print(f"Impuesto total: S/ {impuesto:,.2f}")
        print(f"Tasa efectiva: {tasa_efectiva:.2f}%")
        print(f"Ingreso neto: S/ {ingreso_neto:,.2f}")
        print(f"{'='*60}")

        return impuesto

    except Exception as e:
        return f"Error en el cálculo: {e}"

# Ejemplos de uso
print("Caso 1: Ingreso medio")
calcular_impuesto_renta(80000)

print("\n" + "="*80 + "\n")

print("Caso 2: Ingreso alto")
calcular_impuesto_renta(300000)
```

```
Caso 1: Ingreso medio
CÁLCULO DE IMPUESTO A LA RENTA
============================================================

Ingreso bruto anual: S/ 80,000.00
UIT vigente: S/ 5,150.00

Desglose por tramos:
------------------------------------------------------------

Tramo 1 (0-5 UIT)
  Base imponible: S/ 25,750.00
  Tasa: 8%
```

```
  Impuesto: S/ 2,060.00

Tramo 2 (5-20 UIT)
  Base imponible: S/ 54,250.00
  Tasa: 14%
  Impuesto: S/ 7,595.00

============================================================
Impuesto total: S/ 9,655.00
Tasa efectiva: 12.07%
Ingreso neto: S/ 70,345.00
============================================================

================================================================================

Caso 2: Ingreso alto
CÁLCULO DE IMPUESTO A LA RENTA
============================================================

Ingreso bruto anual: S/ 300,000.00
UIT vigente: S/ 5,150.00

Desglose por tramos:
------------------------------------------------------------

Tramo 1 (0-5 UIT)
  Base imponible: S/ 25,750.00
  Tasa: 8%
  Impuesto: S/ 2,060.00

Tramo 2 (5-20 UIT)
  Base imponible: S/ 77,250.00
  Tasa: 14%
  Impuesto: S/ 10,815.00

Tramo 3 (20-35 UIT)
  Base imponible: S/ 77,250.00
  Tasa: 17%
  Impuesto: S/ 13,132.50

Tramo 4 (35-45 UIT)
  Base imponible: S/ 51,500.00
  Tasa: 20%
  Impuesto: S/ 10,300.00

Tramo 5 (>45 UIT)
  Base imponible: S/ 68,250.00
  Tasa: 30%
```

```
  Impuesto: S/ 20,475.00

============================================================
Impuesto total: S/ 56,782.50
Tasa efectiva: 18.93%
Ingreso neto: S/ 243,217.50
============================================================

56782.5
```

## 12 Ejercicios prácticos

### 12.1 Ejercicio 1: Sistema de evaluación de proyectos de inversión

```python
def evaluar_proyecto(datos_proyecto):
    """
    Evalúa la viabilidad de un proyecto de inversión
    """

    print("EVALUACIÓN DE PROYECTO DE INVERSIÓN")
    print("=" * 60)

    try:
        # Extraer datos
        inversion_inicial = datos_proyecto['inversion_inicial']
        flujos_anuales = datos_proyecto['flujos_anuales']
        tasa_descuento = datos_proyecto['tasa_descuento']
        nombre = datos_proyecto.get('nombre', 'Proyecto sin nombre')

        print(f"\nProyecto: {nombre}")
        print(f"Inversión inicial: S/ {inversion_inicial:,.2f}")
        print(f"Tasa de descuento: {tasa_descuento*100:.1f}%")

        # Calcular VPN
        vpn = -inversion_inicial
        print(f"\nFlujos de caja:")

        for i, flujo in enumerate(flujos_anuales, 1):
            valor_presente = flujo / ((1 + tasa_descuento) ** i)
            vpn += valor_presente
            print(f"  Año {i}: S/ {flujo:,.2f} (VP: S/ {valor_presente:,.2f})")

        # Calcular TIR (aproximación simple)
        # Para cálculo exacto se requeriría método numérico

        # Período de recuperación
        saldo_acumulado = -inversion_inicial
        periodo_recuperacion = None
```

```
        print(f"\nAnálisis de recuperación:")
        for i, flujo in enumerate(flujos_anuales, 1):
            saldo_acumulado += flujo
            print(f"  Año {i}: Saldo acumulado S/ {saldo_acumulado:,.2f}")

            if saldo_acumulado >= 0 and periodo_recuperacion is None:
                periodo_recuperacion = i

        # Evaluación
        print(f"\n{'='*60}")
        print(f"RESULTADOS:")
        print(f"VPN: S/ {vpn:,.2f}")

        if vpn > 0:
            print(f"Recomendación: ACEPTAR el proyecto")
            print(f"El proyecto genera valor")
        elif vpn == 0:
            print(f"Recomendación: INDIFERENTE")
            print(f"El proyecto no genera ni destruye valor")
        else:
            print(f"Recomendación: RECHAZAR el proyecto")
            print(f"El proyecto destruye valor")

        if periodo_recuperacion:
            print(f"\nPeríodo de recuperación: {periodo_recuperacion} años")
        else:
            print(f"\nEl proyecto no recupera la inversión en el período analizado")

        print(f"{'='*60}")

        return vpn

    except KeyError as e:
        print(f"Error: Falta el campo {e}")
        return None
    except Exception as e:
        print(f"Error en la evaluación: {e}")
        return None

# Ejemplo de uso
proyecto_a = {
    'nombre': 'Ampliación de planta',
    'inversion_inicial': 500000,
    'flujos_anuales': [150000, 180000, 200000, 220000, 200000],
    'tasa_descuento': 0.12
}
```

```
vpn_a = evaluar_proyecto(proyecto_a)

print("\n" + "="*80 + "\n")

proyecto_b = {
    'nombre': 'Nueva línea de productos',
    'inversion_inicial': 800000,
    'flujos_anuales': [100000, 150000, 200000, 250000, 300000],
    'tasa_descuento': 0.12
}

vpn_b = evaluar_proyecto(proyecto_b)
```

```
EVALUACIÓN DE PROYECTO DE INVERSIÓN
============================================================

Proyecto: Ampliación de planta
Inversión inicial: S/ 500,000.00
Tasa de descuento: 12.0%

Flujos de caja:
  Año 1: S/ 150,000.00 (VP: S/ 133,928.57)
  Año 2: S/ 180,000.00 (VP: S/ 143,494.90)
  Año 3: S/ 200,000.00 (VP: S/ 142,356.05)
  Año 4: S/ 220,000.00 (VP: S/ 139,813.98)
  Año 5: S/ 200,000.00 (VP: S/ 113,485.37)

Análisis de recuperación:
  Año 1: Saldo acumulado S/ -350,000.00
  Año 2: Saldo acumulado S/ -170,000.00
  Año 3: Saldo acumulado S/ 30,000.00
  Año 4: Saldo acumulado S/ 250,000.00
  Año 5: Saldo acumulado S/ 450,000.00

============================================================
RESULTADOS:
VPN: S/ 173,078.87
Recomendación: ACEPTAR el proyecto
El proyecto genera valor

Período de recuperación: 3 años
============================================================

================================================================================

EVALUACIÓN DE PROYECTO DE INVERSIÓN
============================================================
```

```
Proyecto: Nueva línea de productos
Inversión inicial: S/ 800,000.00
Tasa de descuento: 12.0%

Flujos de caja:
  Año 1: S/ 100,000.00 (VP: S/ 89,285.71)
  Año 2: S/ 150,000.00 (VP: S/ 119,579.08)
  Año 3: S/ 200,000.00 (VP: S/ 142,356.05)
  Año 4: S/ 250,000.00 (VP: S/ 158,879.52)
  Año 5: S/ 300,000.00 (VP: S/ 170,228.06)

Análisis de recuperación:
  Año 1: Saldo acumulado S/ -700,000.00
  Año 2: Saldo acumulado S/ -550,000.00
  Año 3: Saldo acumulado S/ -350,000.00
  Año 4: Saldo acumulado S/ -100,000.00
  Año 5: Saldo acumulado S/ 200,000.00

============================================================
RESULTADOS:
VPN: S/ -119,671.58
Recomendación: RECHAZAR el proyecto
El proyecto destruye valor

Período de recuperación: 5 años
============================================================
```

### 12.2 Ejercicio 2: Simulador de política monetaria

```python
def simular_politica_monetaria(estado_economia):
    """
    Simula decisiones de política monetaria según el estado de la economía
    """

    print("SIMULADOR DE POLÍTICA MONETARIA")
    print("=" * 60)

    try:
        # Extraer indicadores
        inflacion_actual = estado_economia['inflacion_actual']
        inflacion_esperada = estado_economia['inflacion_esperada']
        meta_inflacion = estado_economia['meta_inflacion']
        brecha_producto = estado_economia['brecha_producto']
        tasa_actual = estado_economia['tasa_interes_actual']

        print(f"\nIndicadores macroeconómicos:")
        print(f"  Inflación actual: {inflacion_actual:.2f}%")
        print(f"  Inflación esperada: {inflacion_esperada:.2f}%")
```

```
        print(f"  Meta de inflación: {meta_inflacion:.2f}%")
        print(f"  Brecha del producto: {brecha_producto:+.2f}%")
        print(f"  Tasa de interés actual: {tasa_actual:.2f}%")

        # Regla de Taylor simplificada
        # r = r* +   + 0.5(  -  *) + 0.5(y)
        # donde:
        # r = tasa de interés nominal
        # r* = tasa de interés real neutral (asumimos 2%)
        #   = inflación
        #  * = meta de inflación
        # y = brecha del producto

        tasa_neutral = 2.0
        desviacion_inflacion = inflacion_esperada - meta_inflacion

        # Calcular tasa sugerida según Regla de Taylor
        tasa_sugerida = tasa_neutral + inflacion_esperada + \
                        0.5 * desviacion_inflacion + \
                        0.5 * brecha_producto

        # Determinar acción
        diferencia = tasa_sugerida - tasa_actual

        print(f"\n{'='*60}")
        print(f"ANÁLISIS:")

        # Evaluar presiones inflacionarias
        if inflacion_actual > meta_inflacion + 1.0:
            print(f"   Inflación significativamente por encima de la meta")
            presion_inflacionaria = "alta"
        elif inflacion_actual > meta_inflacion:
            print(f"  Inflación ligeramente por encima de la meta")
            presion_inflacionaria = "moderada"
        elif inflacion_actual < meta_inflacion - 1.0:
            print(f"  Inflación significativamente por debajo de la meta")
            presion_inflacionaria = "baja"
        else:
            print(f"  Inflación dentro del rango meta")
            presion_inflacionaria = "normal"

        # Evaluar actividad económica
        if brecha_producto > 2.0:
            print(f"   Economía sobrecalentada")
            presion_demanda = "alta"
        elif brecha_producto > 0:
            print(f"  Economía por encima de su potencial")
```

```
            presion_demanda = "moderada"
        elif brecha_producto < -2.0:
            print(f"   Economía en recesión")
            presion_demanda = "muy baja"
        else:
            print(f"  Economía por debajo de su potencial")
            presion_demanda = "baja"

        # Recomendación
        print(f"\n{'='*60}")
        print(f"RECOMENDACIÓN DE POLÍTICA:")

        if abs(diferencia) < 0.25:
            decision = "MANTENER"
            tasa_nueva = tasa_actual
            print(f"  MANTENER la tasa de interés en {tasa_actual:.2f}%")
            print(f"  La tasa actual es adecuada para las condiciones macroeconómicas

        elif diferencia > 0:
            if abs(diferencia) > 0.75:
                ajuste = 0.50
                decision = "SUBIR FUERTE"
            else:
                ajuste = 0.25
                decision = "SUBIR"

            tasa_nueva = tasa_actual + ajuste
            print(f"↑ {decision} la tasa de interés a {tasa_nueva:.2f}%")
            print(f"   Ajuste: +{ajuste:.2f} puntos porcentuales")

            if presion_inflacionaria in ["alta", "moderada"]:
                print(f"   Justificación: Controlar presiones inflacionarias")
            if presion_demanda == "alta":
                print(f"   Justificación: Enfriar la economía sobrecalentada")

        else:
            if abs(diferencia) > 0.75:
                ajuste = -0.50
                decision = "BAJAR FUERTE"
            else:
                ajuste = -0.25
                decision = "BAJAR"

            tasa_nueva = tasa_actual + ajuste
            print(f"↓ {decision} la tasa de interés a {tasa_nueva:.2f}%")
            print(f"   Ajuste: {ajuste:.2f} puntos porcentuales")
```

```
            if presion_inflacionaria == "baja":
                print(f"  Justificación: Evitar presiones deflacionarias")
            if presion_demanda in ["baja", "muy baja"]:
                print(f"  Justificación: Estimular la actividad económica")

        # Efectos esperados
        print(f"\nEfectos esperados:")
        if decision in ["SUBIR", "SUBIR FUERTE"]:
            print(f"  • Encarecimiento del crédito")
            print(f"  • Reducción del consumo e inversión")
            print(f"  • Desaceleración de la inflación")
            print(f"  • Apreciación de la moneda local")
        elif decision in ["BAJAR", "BAJAR FUERTE"]:
            print(f"  • Abaratamiento del crédito")
            print(f"  • Estímulo al consumo e inversión")
            print(f"  • Impulso a la actividad económica")
            print(f"  • Depreciación de la moneda local")
        else:
            print(f"  • Mantenimiento de las condiciones actuales")
            print(f"  • Monitoreo continuo de indicadores")

        print(f"{'='*60}")

        return {
            'decision': decision,
            'tasa_nueva': tasa_nueva,
            'tasa_sugerida_taylor': tasa_sugerida
        }

    except Exception as e:
        print(f"Error en la simulación: {e}")
        return None

# Escenario 1: Inflación alta
print("ESCENARIO 1: Presiones inflacionarias")
escenario_1 = {
    'inflacion_actual': 5.2,
    'inflacion_esperada': 5.5,
    'meta_inflacion': 3.0,
    'brecha_producto': 1.5,
    'tasa_interes_actual': 5.5
}
resultado_1 = simular_politica_monetaria(escenario_1)

print("\n" + "="*80 + "\n")

# Escenario 2: Recesión
```

```
print("ESCENARIO 2: Recesión económica")
escenario_2 = {
    'inflacion_actual': 2.0,
    'inflacion_esperada': 1.8,
    'meta_inflacion': 3.0,
    'brecha_producto': -3.5,
    'tasa_interes_actual': 4.0
}
resultado_2 = simular_politica_monetaria(escenario_2)
```

```
ESCENARIO 1: Presiones inflacionarias
SIMULADOR DE POLÍTICA MONETARIA
============================================================

Indicadores macroeconómicos:
  Inflación actual: 5.20%
  Inflación esperada: 5.50%
  Meta de inflación: 3.00%
  Brecha del producto: +1.50%
  Tasa de interés actual: 5.50%

============================================================
ANÁLISIS:
   Inflación significativamente por encima de la meta
  Economía por encima de su potencial

============================================================
RECOMENDACIÓN DE POLÍTICA:
↑ SUBIR FUERTE la tasa de interés a 6.00%
  Ajuste: +0.50 puntos porcentuales
  Justificación: Controlar presiones inflacionarias

Efectos esperados:
  • Encarecimiento del crédito
  • Reducción del consumo e inversión
  • Desaceleración de la inflación
  • Apreciación de la moneda local
============================================================

================================================================================

ESCENARIO 2: Recesión económica
SIMULADOR DE POLÍTICA MONETARIA
============================================================

Indicadores macroeconómicos:
  Inflación actual: 2.00%
  Inflación esperada: 1.80%
```

```
  Meta de inflación: 3.00%
  Brecha del producto: -3.50%
  Tasa de interés actual: 4.00%

============================================================
ANÁLISIS:
  Inflación dentro del rango meta
   Economía en recesión

============================================================
RECOMENDACIÓN DE POLÍTICA:
↓ BAJAR FUERTE la tasa de interés a 3.50%
  Ajuste: -0.50 puntos porcentuales
  Justificación: Estimular la actividad económica

Efectos esperados:
  • Abaratamiento del crédito
  • Estímulo al consumo e inversión
  • Impulso a la actividad económica
  • Depreciación de la moneda local
============================================================
```

## 13 Conclusión

En esta guía hemos explorado la ejecución condicional en Python:

**Expresiones booleanas**

Expresiones que evalúan a True o False. Son la base de la toma de decisiones en programación.

**Operadores de comparación**

Permiten comparar valores: `==`, `!=`, `>`, `<`, `>=`, `<=`.

**Operadores lógicos**

Combinan expresiones booleanas: `and`, `or`, `not`. La precedencia es: `not` > `and` > `or`.

**Estructuras condicionales**

`if` para ejecución simple, `if-else` para alternativas, `if-elif-else` para múltiples condiciones.

**Condicionales anidados**

Permiten lógica compleja pero deben usarse con moderación para mantener legibilidad.

**Manejo de excepciones**

`try-except` permite capturar y manejar errores sin detener el programa.

**Evaluación de cortocircuito**

Python evalúa expresiones booleanas eficientemente, deteniéndose cuando el resultado es determinado.

## 14 Próximos pasos

En la siguiente guía (Guía 5: Iteraciones) exploraremos:

- Actualización de variables
- Bucles definidos con for
- Iteración doble, múltiple y anidada

- Bucles while
- List comprehension

Las iteraciones nos permitirán procesar grandes cantidades de datos económicos de manera eficiente y automatizada.

## 15 Recursos adicionales

Para profundizar en ejecución condicional:

- Python Control Flow: docs.python.org/tutorial/controlflow.html
- Práctica con casos económicos reales
- Estudio de algoritmos de decisión en economía

## 16 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python
5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python
8. 06 Funciones Con Python
9. 07 Dataframes Con Python
10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!